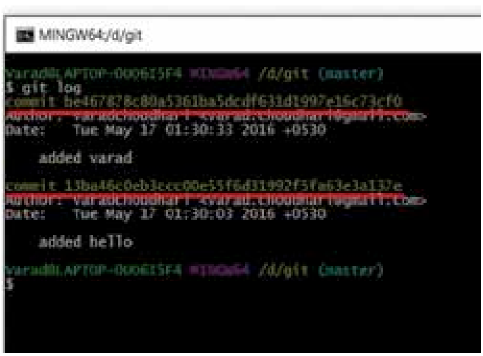
Viewing old commits

The log command will list your commit history. If you want to see the code in the previous commit without altering the current modification you can use the command **"git checkout <commit> <filename>"**. The commit in the above command is the unique commit ID that you'll see above every commit listed in the commit history. The checkout command makes your working directory match the exact state of the old commit. You can do anything you want including compile, modify, rerun, etc.

Undoing changes

If you want to undo the last commit made you can use **"git revert <commit>"**. Where the <commit> is the unique commit ID. You can permanently undo your commit by using **"git reset"**. Care must be taken while using this command as you are not in the position to get back the data. The command **"git reset <file>"** will remove the specified file from the staging area, but leave the working directory unchanged. The command **"git reset"** removes all the files from the staging area to match the last commit made. The nuclear weapon here is **"git reset --hard"** which clears the staging area and rolls back the working directory to the last commit made.



The unique commit located above the author name should be used for "checkout".

Cloning projects or modules from GitHub

We talked about working on the local directory where all the changes made to the code were hardcoded. You are already aware about GitHub – a public Git repository from where you can get source codes for your project, and also share yours. Cloning a repo is different than forking it. Forking makes you a contributor of the original project, whereas, cloning downloads the project which you can use as a part of your existing project. Cloning an existing project from the GitHub to your local repository is relatively easy.